

Experiment-3

Water Jug Problem

Aim

To develop a Python program to solve the Water Jug Problem using the Breadth-First Search (BFS) algorithm.

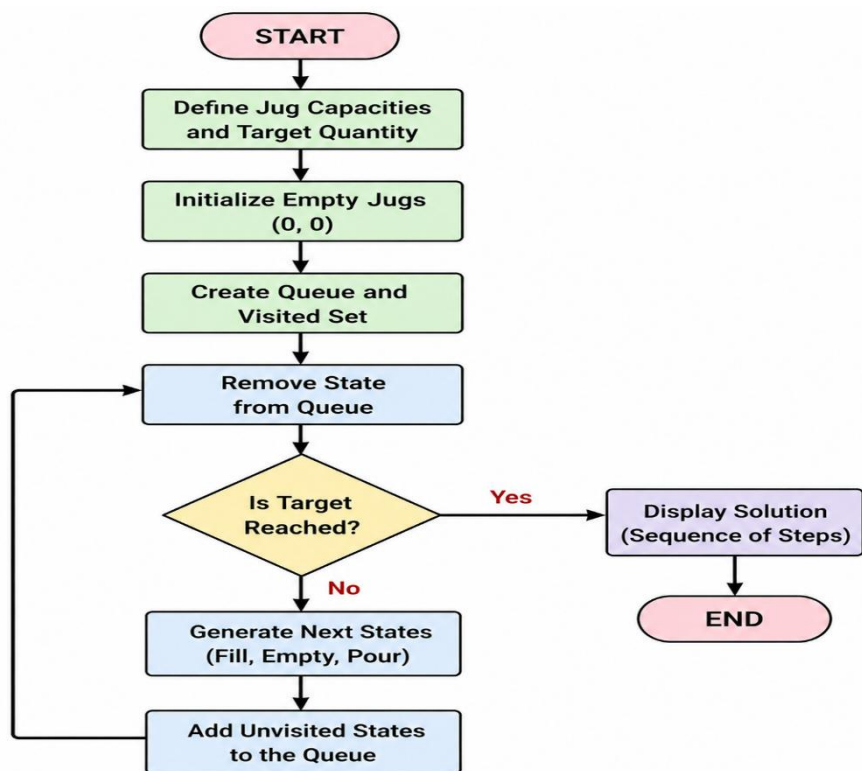
Objective

- To solve the Water Jug Problem using BFS.
- To obtain the desired amount of water using two jugs of different capacities.
- To understand state-space search techniques in Artificial Intelligence.

Algorithm

1. Start the program.
2. Define the capacities of the two water jugs and the target quantity.
3. Initialize the starting state with both jugs empty.
4. Create a queue and a visited set.
5. Remove a state from the queue.
6. Check whether the target quantity is reached.
7. If reached, display the solution and stop.
8. Otherwise, generate all possible next states by filling, emptying, or pouring water between the jugs.
9. Add unvisited states to the queue.
10. Repeat until the target is found or all states are explored.
11. Stop the program.

Flowchart:



Code:

```
from collections import deque
```

```
def water_jug():
```

```
    q = deque([((0, 0), [])])
```

```
    visited = set()
```

```
    while q:
```

```
        (a, b), path = q.popleft()
```

```
        if (a, b) in visited:
```

```
            continue
```

```
        visited.add((a, b))
```

```
        path = path + [(a, b)]
```

```
if a == 2 or b == 2:
```

```
    return path
```

```
next_states = [
```

```
    (4, b),      # Fill 4L jug
```

```
    (a, 3),      # Fill 3L jug
```

```
    (0, b),      # Empty 4L jug
```

```
    (a, 0),      # Empty 3L jug
```

```
    (max(0, a-(3-b)), min(3, a+b)), # Pour 4L -> 3L
```

```
    (min(4, a+b), max(0, b-(4-a))) # Pour 3L -> 4L
```

```
]
```

```
for s in next_states:
```

```
    if s not in visited:
```

```
        q.append((s, path))
```

```
return None
```

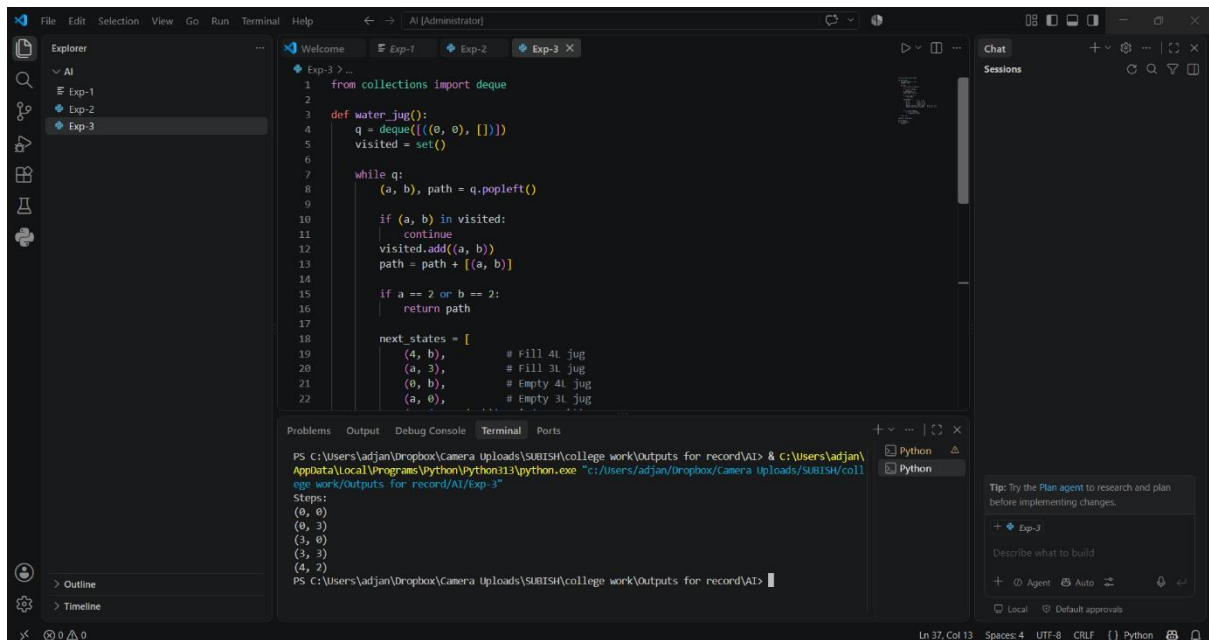
```
solution = water_jug()
```

```
print("Steps:")
```

```
for s in solution:
```

```
    print(s)
```

Output:



```
1 from collections import deque
2
3 def water_jug():
4     q = deque([(0, 0), []])
5     visited = set()
6
7     while q:
8         (a, b), path = q.popleft()
9
10        if (a, b) in visited:
11            continue
12        visited.add((a, b))
13        path = path + [(a, b)]
14
15        if a == 2 or b == 2:
16            return path
17
18        next_states = [
19            (4, b),      # Fill 4L jug
20            (a, 3),      # Fill 3L jug
21            (0, b),      # Empty 4L jug
22            (a, 0),      # Empty 3L jug
```

PS C:\Users\adjan\Dropbox\Camera Uploads\SUBISH\college work\Outputs for record\AI> & C:\Users\adjan\AppData\Local\Programs\Python\python313\python.exe "C:\Users\adjan\Dropbox\Camera Uploads\SUBISH\college work\Outputs for record\AI\Exp-3"

Steps:

- (0, 0)
- (0, 3)
- (3, 0)
- (3, 3)
- (4, 2)

Result:

The Water Jug Problem was successfully solved using the Breadth-First Search (BFS) algorithm, and the required quantity of water was obtained.

Conclusion:

The Breadth-First Search (BFS) algorithm efficiently solved the Water Jug Problem by finding the shortest sequence of operations to reach the target state.